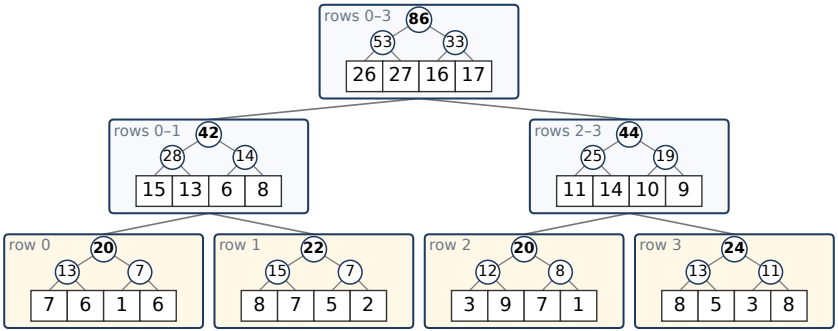
Multidimensional Search Structures I

Range trees

AIL7002 · Data Structures and Algorithms (refresher)

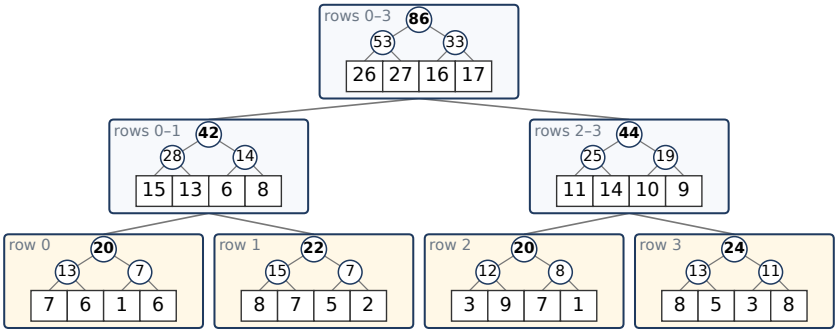
IIT Delhi

The two-dimensional segment tree



Outer tree over the rows; each node holds an **inner tree** over the columns, on the column sums of its rows. Sum query $O(\log^2 n)$, memory $O(n^2)$.

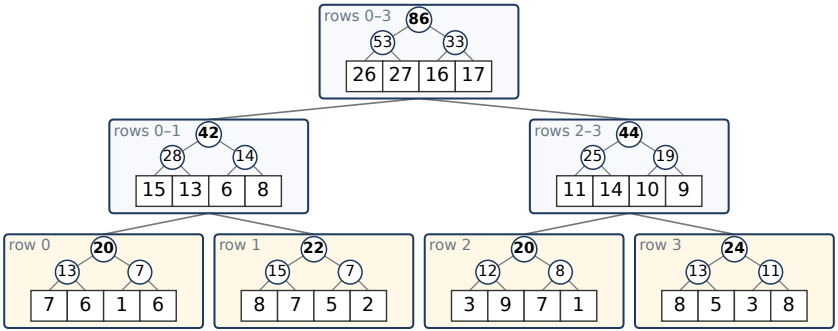
The two-dimensional segment tree



Outer tree over the rows; each node holds an **inner tree** over the columns, on the column sums of its rows. Sum query $O(\log^2 n)$, memory $O(n^2)$.

Today: N points, arbitrary coordinates; report the points in a box.

The two-dimensional segment tree



Outer tree over the rows; each node holds an **inner tree** over the columns, on the column sums of its rows. Sum query $O(\log^2 n)$, memory $O(n^2)$.

Today: N points, arbitrary coordinates; report the points in a box.

Same skeleton, new payload: outer tree over the x-order, y-sorted lists in its nodes.

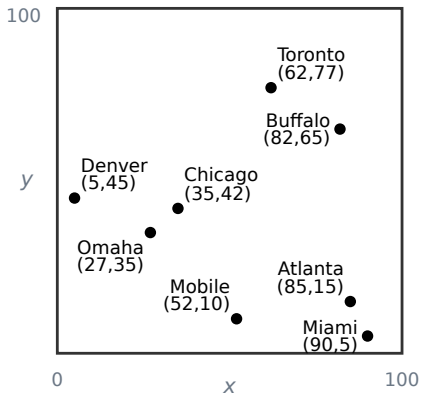
Multidimensional point data

A record with d keys (attributes) is a point in d-dimensional space (Samet p. 1): an employee (age, height, salary) is a point in $\mathbb{R}^3$.

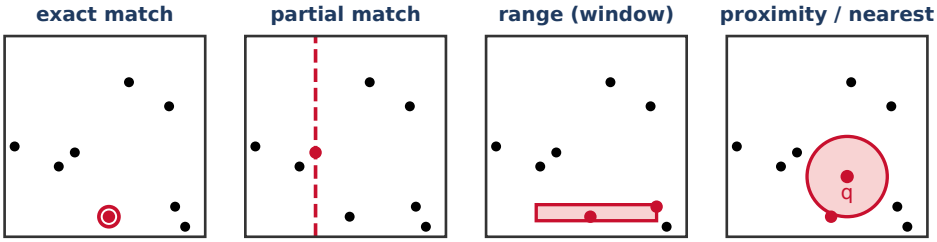
Running example, used for every structure in both lectures (Fig. 1.1):

Name	x	y	Name	x	y
Chicago	35	42	Denver	5	45
Mobile	52	10	Omaha	27	35
Toronto	62	77	Atlanta	85	15
Buffalo	82	65	Miami	90	5

Insertion order: the table, column by column.



What do we ask?



- **Exact match** (point query): is *p* in the set?
- **Partial match**: some keys given exactly, the others free. **Range query**: an interval per key (an axis-parallel box).
- **Proximity**: within distance *r*, or the nearest neighbour – meaningful only when all keys share a unit (distance).

Design questions (Samet pp. 1–3)

1. What type of data: discrete or continuous, bounded domain?
2. Which operations (queries, updates)?
3. Organize the data or the embedding space?
4. Static or dynamic?
5. Main memory or disk?

Question 3, in one dimension:

- binary search tree: split values come from the data (*tree-based*);
- trie / radix search: split positions fixed in advance, e.g. halve $[0, 100)$ (*trie-based*); needs a bounded domain; shape independent of insertion order.

In d dimensions: tree-based $\rightarrow$ **range tree, point quadtree, k-d tree**; trie-based $\rightarrow$ grid, MX/PR quadtree, PR k-d tree.

Baselines

Sequential scan and inverted lists (Samet p. 5)

Sequential list: check every record, $O(N \cdot d)$ per query.

Inverted file: one sorted list per key (pointers to the records).

- binary search on one key (the *primary*) narrows to a candidate list,
- then check the other keys by brute force;
- or query every list and intersect – wasteful.

Average cost $O(N^{1-1/d})$ under uniformity assumptions (Friedman, Baskett & Shustek) – next slide.

Sequential scan and inverted lists (Samet p. 5)

Sequential list: check every record, $O(N \cdot d)$ per query.

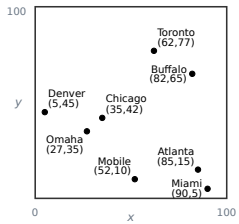
Inverted file: one sorted list per key (pointers to the records).

- binary search on one key (the *primary*) narrows to a candidate list,
- then check the other keys by brute force;
- or query every list and intersect – wasteful.

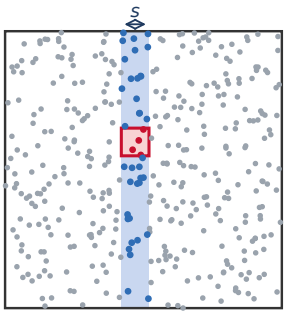
Average cost $O(N^{1-1/d})$ under uniformity assumptions (Friedman, Baskett & Shustek) – next slide.

Example $[25 : 85] \times [8 : 16]$: the x-list gives 6 candidates, only 2 survive.

sorted by x	sorted by y
Denver	Miami
Omaha	Mobile
Chicago	Atlanta
Mobile	Omaha
Toronto	Chicago
Buffalo	Denver
Atlanta	Buffalo
Miami	Toronto



Why $O(N^{1-\frac{1}{d}})$? One key prunes only one dimension

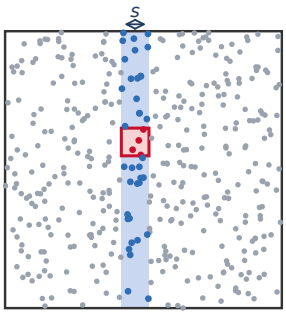


$N = 400$ uniform points; query square with $F = 4$. The x-list returns the whole slab: 40 points.

N points uniform in $[0,1]^d$; a small query cube holding $\approx F$ points.

1. Volume F/N, so side $s = (F/N)^{1/d}$.

Why $O(N^{1-\frac{1}{d}})$? One key prunes only one dimension

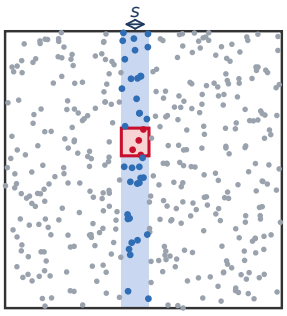


$N = 400$ uniform points; query square with $F = 4$. The x-list returns the whole slab: 40 points.

N points uniform in $[0,1]^d$; a small query cube holding $\approx F$ points.

1. Volume F/N, so side $s = (F/N)^{1/d}$.
2. Binary search on x returns the slab of width s, unrestricted in the other $d-1$ keys – all of it is checked by brute force: $N \cdot s = F^{1/d}\, N^{1-1/d}$.

Why $O(N^{1-\frac{1}{d}})$? One key prunes only one dimension

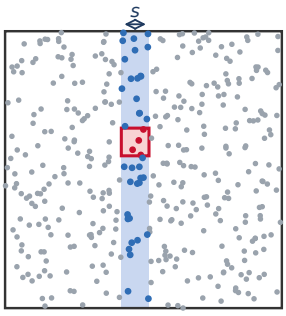


$N = 400$ uniform points; query square with $F = 4$. The x-list returns the whole slab: 40 points.

N points uniform in $[0,1]^d$; a small query cube holding $\approx F$ points.

1. Volume F/N, so side $s = (F/N)^{1/d}$.
2. Binary search on x returns the slab of width s, unrestricted in the other $d-1$ keys – all of it is checked by brute force: $N \cdot s = F^{1/d}\, N^{1-1/d}$.
3. The slab holds $(1/s)^{d-1} = (N/F)^{(d-1)/d}$ times more points than the answer.

Why $O(N^{1-\frac{1}{d}})$? One key prunes only one dimension



$N = 400$ uniform points; query square with $F = 4$. The x-list returns the whole slab: 40 points.

N points uniform in $[0,1]^d$; a small query cube holding $\approx F$ points.

1. Volume F/N, so side $s = (F/N)^{1/d}$.
2. Binary search on x returns the slab of width s, unrestricted in the other $d-1$ keys – all of it is checked by brute force: $N \cdot s = F^{1/d}\, N^{1-1/d}$.
3. The slab holds $(1/s)^{d-1} = (N/F)^{(d-1)/d}$ times more points than the answer.

$N = 10^6$, $F \approx 1$	$d = 2$	$d = 3$	$d = 10$
candidates	$10^3 = \sqrt{N}$	$10^4 = N^{2/3}$	$N^{0.9} \approx 2.5 \times 10^5$

Average case for *small* queries; a large box makes the slab large anyway.

Combining the keys (Samet pp. 5–9)

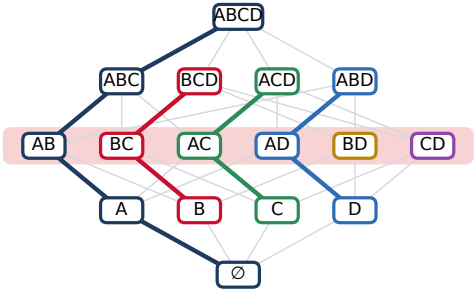
- **Combined index**: sort lexicographically on $(A_1, A_2, \dots)$ – one "super-key". Good only when the leading keys are exact. Covering every partial-match pattern needs $\binom{d}{\lfloor (d+1)/2 \rfloor}$ such indexes (3 for $d = 3$: ABC, BCA, CAB) – next slide.
- **Multiattribute tree**: one level per key, each level a search structure on that key (with B-trees: the multidimensional B-tree, kB-tree).
- The multidimensional range tree and the priority search tree are in this family – with a guaranteed worst case.

Combining the keys (Samet pp. 5–9)

- **Combined index**: sort lexicographically on $(A_1, A_2, \dots)$ – one "super-key". Good only when the leading keys are exact. Covering every partial-match pattern needs $\binom{d}{\lfloor (d+1)/2 \rfloor}$ such indexes (3 for $d = 3$: ABC, BCA, CAB) – next slide.
- **Multiattribute tree**: one level per key, each level a search structure on that key (with B-trees: the multidimensional B-tree, kB-tree).
- The multidimensional range tree and the priority search tree are in this family – with a guaranteed worst case.

Two families of solutions (Samet p. 6): (1) a hierarchy of one-dimensional structures, one key after another; (2) a partition of space into d-dimensional cells (grids, quadtrees, k-d trees). Today: mostly family (1).

Why $\binom{d}{\lfloor d/2 \rfloor}$ combined indexes? (Lum 1970; Samet p. 6)



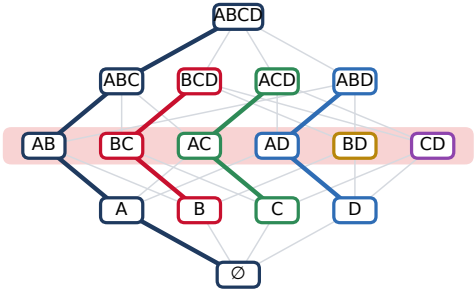
All 16 subsets of $\{A, B, C, D\}$; coloured paths = 6 chains covering them all. Shaded: the 6 sets of size 2.

ABCD BCDA CADB DABC BDAC CDAB

d	2	3	4	5	6	10
indexes	2	3	6	10	20	252
$\approx 2^d/\sqrt{\pi d/2}$						

1. An index sorted on $A_1A_2 \ldots A_d$ returns one contiguous run only if the given keys are a *prefix* $\{A_1, \ldots, A_i\}$.

Why $\binom{d}{\lfloor d/2 \rfloor}$ combined indexes? (Lum 1970; Samet p. 6)



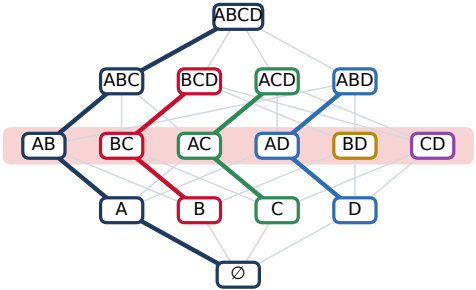
All 16 subsets of $\{A, B, C, D\}$; coloured paths = 6 chains covering them all. Shaded: the 6 sets of size 2.

ABCD BCDA CADB DABC BDAC CDAB

d	2	3	4	5	6	10
indexes	2	3	6	10	20	252
$\approx 2^d/\sqrt{\pi d/2}$						

1. An index sorted on $A_1 A_2 \ldots A_d$ returns one contiguous run only if the given keys are a *prefix* $\{A_1, \ldots, A_i\}$.
2. So one index covers a chain $\emptyset \subset \{A_1\} \subset \{A_1, A_2\} \subset \ldots$ (one set per size); the chains must cover all 2^d subsets.

Why $\binom{d}{\lfloor d/2 \rfloor}$ combined indexes? (Lum 1970; Samet p. 6)



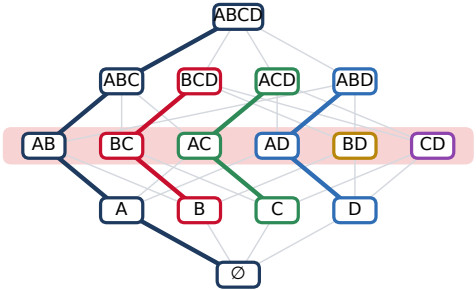
All 16 subsets of $\{A, B, C, D\}$; coloured paths = 6 chains covering them all. Shaded: the 6 sets of size 2.

ABCD BCDA CADB DABC BDAC CDAB

d	2	3	4	5	6	10
indexes	2	3	6	10	20	252
$\approx 2^d/\sqrt{\pi d/2}$						

1. An index sorted on $A_1 A_2 \ldots A_d$ returns one contiguous run only if the given keys are a *prefix* $\{A_1, \ldots, A_i\}$.
2. So one index covers a chain $\emptyset \subset \{A_1\} \subset \{A_1, A_2\} \subset \ldots$ (one set per size); the chains must cover all 2^d subsets.
3. **Lower bound**: two sets of equal size never share a chain $\Rightarrow$ at least $\binom{d}{\lfloor d/2 \rfloor}$ indexes (largest layer).

Why $\binom{d}{\lfloor d/2 \rfloor}$ combined indexes? (Lum 1970; Samet p. 6)



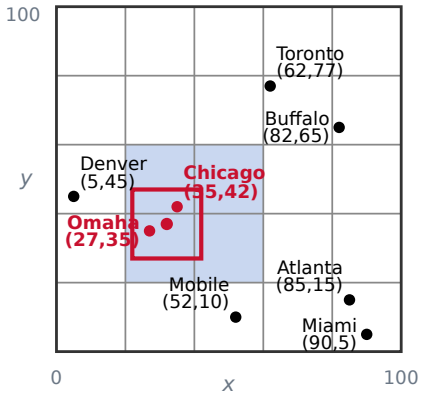
All 16 subsets of $\{A, B, C, D\}$; coloured paths = 6 chains covering them all. Shaded: the 6 sets of size 2.

ABCD BCDA CADB DABC BDAC CDAB

d	2	3	4	5	6	10
indexes	2	3	6	10	20	252
$\approx 2^d/\sqrt{\pi d/2}$						

1. An index sorted on $A_1 A_2 \ldots A_d$ returns one contiguous run only if the given keys are a *prefix* $\{A_1, \ldots, A_i\}$.
2. So one index covers a chain $\emptyset \subset \{A_1\} \subset \{A_1, A_2\} \subset \ldots$ (one set per size); the chains must cover all 2^d subsets.
3. **Lower bound**: two sets of equal size never share a chain $\Rightarrow$ at least $\binom{d}{\lfloor d/2 \rfloor}$ indexes (largest layer).
4. **Upper bound**: a symmetric chain decomposition (de Bruijn et al. 1951) achieves it.

The fixed grid (Samet pp. 10–11)

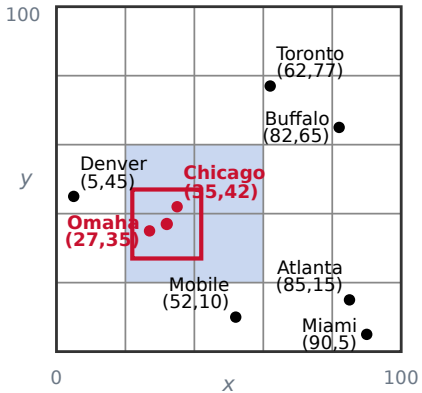


Overlay a grid; each cell holds a list of its points; a d-dimensional array maps a point to its cell in $O(1)$.

Cell side $=$ query side (here 20): a query box meets at most 2^d cells, average time $O(F \cdot 2^d)$ (Bentley et al.).

Query: the 20×20 square centred at $(32, 37) \rightarrow 4$ cells $\rightarrow$ Chicago, Omaha.

The fixed grid (Samet pp. 10–11)



Overlay a grid; each cell holds a list of its points; a d-dimensional array maps a point to its cell in $O(1)$.

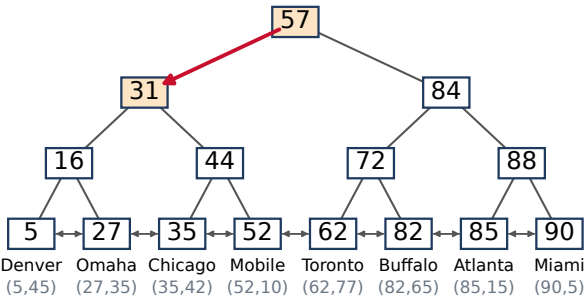
Cell side $=$ query side (here 20): a query box meets at most 2^d cells, average time $O(F \cdot 2^d)$ (Bentley et al.).

Query: the 20×20 square centred at $(32, 37) \rightarrow 4$ cells $\rightarrow$ Chicago, Omaha.

Problem: real data are not uniform – some cells overflow, most are empty. Adapting the cells leads to quadtrees (Next Lecture).

Range trees

One dimension: the 1D range tree (Samet pp. 14–15, Fig. 1.7)



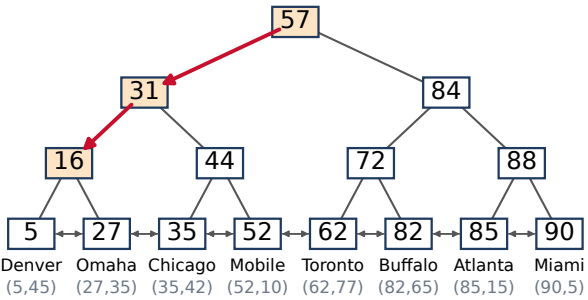
Balanced BST; points only in the leaves, linked in sorted order. Internal nodes hold a MidRange value ($\geq$ left, $\leq$ right).

RangeSearch(B, E): descend to the leaf for B, then follow Next while $\leq E$.

$O(\log N + F)$ time, $O(N)$ space.

`RANGESEARCH(28, 62)` B = 28 ≤ 57? → left

One dimension: the 1D range tree (Samet pp. 14–15, Fig. 1.7)



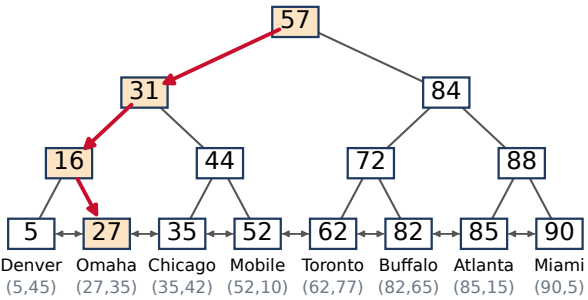
Balanced BST; points only in the leaves, linked in sorted order. Internal nodes hold a MidRange value ($\geq$ left, $\leq$ right).

RangeSearch(B, E): descend to the leaf for B, then follow Next while $\leq E$.

$O(\log N + F)$ time, $O(N)$ space.

`RANGESEARCH(28, 62)` B = 28 ≤ 31? → left

One dimension: the 1D range tree (Samet pp. 14–15, Fig. 1.7)



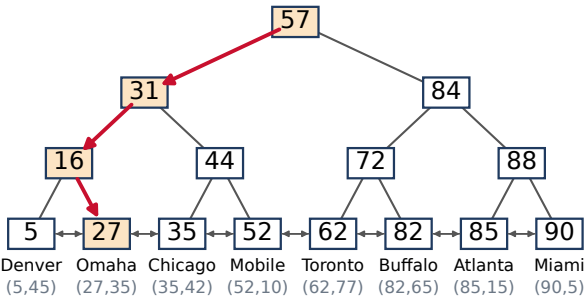
Balanced BST; points only in the leaves, linked in sorted order. Internal nodes hold a MidRange value ($\geq$ left, $\leq$ right).

RangeSearch(B, E): descend to the leaf for B, then follow Next while $\leq E$.

$O(\log N + F)$ time, $O(N)$ space.

`RANGESEARCH(28, 62)` B = 28 ≤ 16? → right

One dimension: the 1D range tree (Samet pp. 14–15, Fig. 1.7)



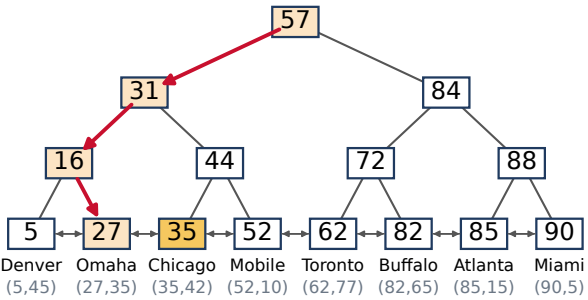
Balanced BST; points only in the leaves, linked in sorted order. Internal nodes hold a MidRange value ($\geq$ left, $\leq$ right).

RangeSearch(B, E): descend to the leaf for B, then follow Next while $\leq E$.

$O(\log N + F)$ time, $O(N)$ space.

`RANGESEARCH(28, 62)` leaf 27 < B: follow NEXT

One dimension: the 1D range tree (Samet pp. 14–15, Fig. 1.7)



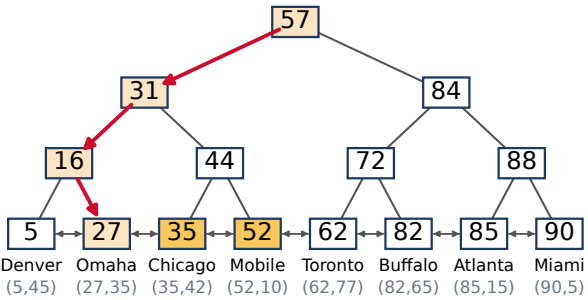
Balanced BST; points only in the leaves, linked in sorted order. Internal nodes hold a MidRange value ($\geq$ left, $\leq$ right).

RangeSearch(B, E): descend to the leaf for B, then follow Next while $\leq E$.

$O(\log N + F)$ time, $O(N)$ space.

`RANGESEARCH(28, 62)` output 35 (Chicago)

One dimension: the 1D range tree (Samet pp. 14–15, Fig. 1.7)



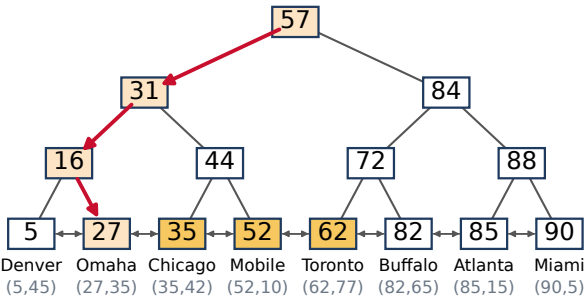
Balanced BST; points only in the leaves, linked in sorted order. Internal nodes hold a MidRange value ($\geq$ left, $\leq$ right).

RangeSearch(B, E): descend to the leaf for B, then follow Next while $\leq E$.

$O(\log N + F)$ time, $O(N)$ space.

`RANGESEARCH(28, 62)` output 52 (Mobile)

One dimension: the 1D range tree (Samet pp. 14–15, Fig. 1.7)



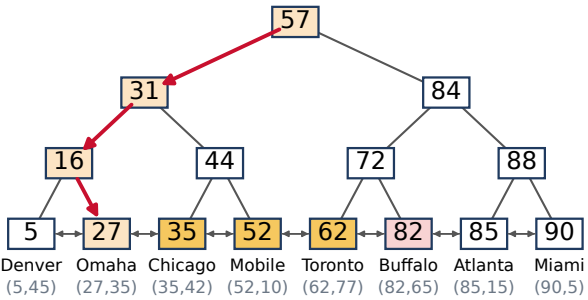
Balanced BST; points only in the leaves, linked in sorted order. Internal nodes hold a MidRange value ($\geq$ left, $\leq$ right).

RangeSearch(B, E): descend to the leaf for B, then follow Next while $\leq E$.

$O(\log N + F)$ time, $O(N)$ space.

`RANGESEARCH(28, 62)` output 62 (Toronto)

One dimension: the 1D range tree (Samet pp. 14–15, Fig. 1.7)



Balanced BST; points only in the leaves, linked in sorted order. Internal nodes hold a MidRange value ($\geq$ left, $\leq$ right).

RangeSearch(B, E): descend to the leaf for B, then follow Next while $\leq E$.

$O(\log N + F)$ time, $O(N)$ space.

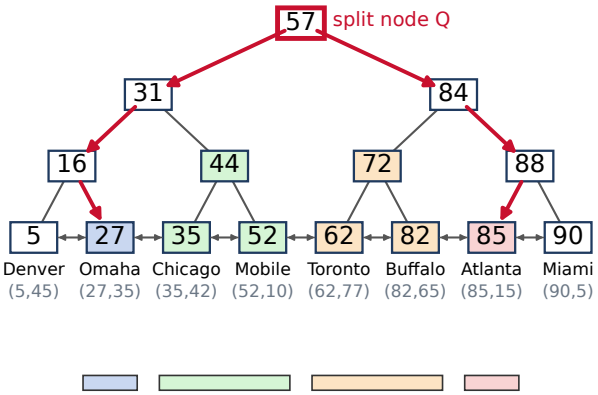
`RANGESEARCH(28, 62)` 82 > E = 62: stop

Static version: a sorted array

```cpp
// 1D range tree, static version: the leaves in sorted order are just a sorted array
vector<int> range1d(const vector<int>& xs, int B, int E) {    // xs sorted
    vector<int> out;
    auto it = lower_bound(xs.begin(), xs.end(), B);   // descend: first leaf >= B
    for (; it != xs.end() && *it <= E; ++it)           // follow the NEXT links
        out.push_back(*it);
    return out;                                         // O(log N + F)
}
```

- The leaves in order *are* a sorted array; `lower_bound` is the descent, `++it` is Next.
- The tree is needed when points are inserted and deleted (balanced BST, e.g. `std::set`) – and, next, because its internal nodes are where 2D information will live.

A range is a union of $O(\log N)$ subtrees

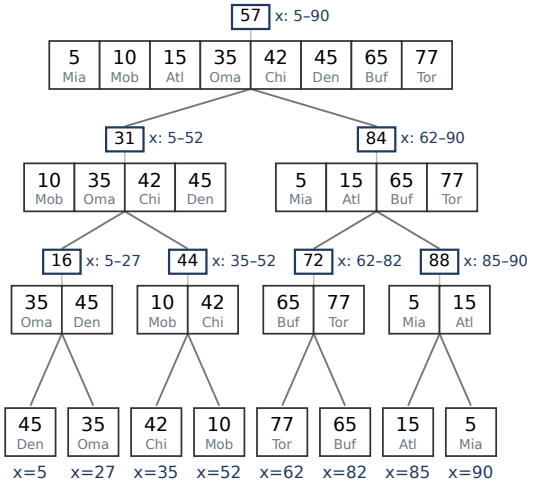


Query $[20 : 86]$. Follow the search paths for 20 and 86 from the split node Q (Samet's "nearest common ancestor").

Left path: whenever it goes left, the right child's subtree is inside. Right path: symmetric.

These *canonical subtrees* (at most 2 per level) are exactly the nodes a segment-tree query visits.

The 2D range tree (Samet pp. 15–16, Fig. 1.8)

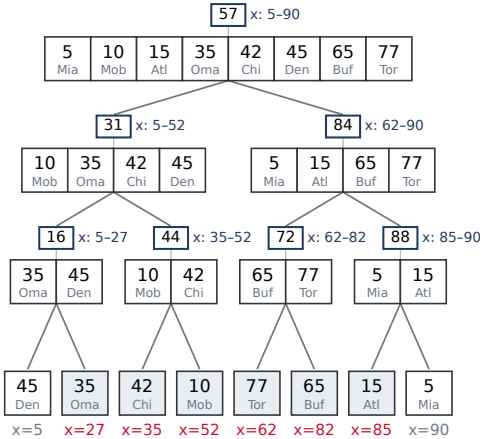


Primary tree: the 1D range tree on x – squares: its split keys; the x-range each node covers is printed beside them.

Every node l also stores its associated structure T_l: the points below l sorted by y (here an array; y-value, city underneath).

A "binary tree of binary trees" – and the same picture as a merge sort: each array is the merge of its two children.

2D range search (Samet pp. 16–18, procedure 2dSearch)



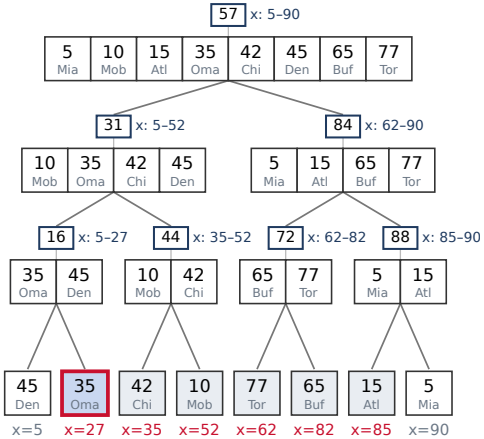
$x \in [25, 85] \rightarrow$ leaves Omaha … Atlanta

Query $[25 : 85] \times [8 : 16]$:

1. search x in the primary tree $\rightarrow O(\log N)$ canonical nodes;
2. in each, a 1D range search on y in its associated structure.

Answer: Mobile, Atlanta – as in the book (p. 18).

2D range search (Samet pp. 16–18, procedure 2dSearch)



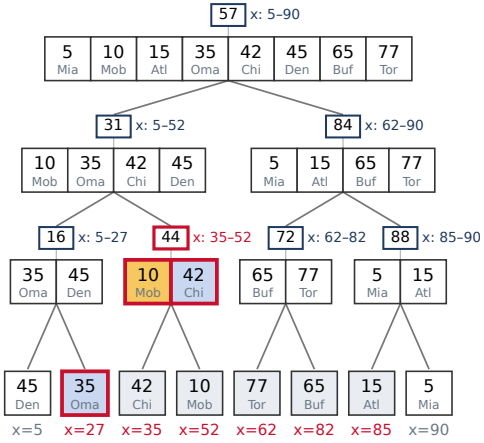
node {Oma}: y ∈ [8,16] → none

Query $[25 : 85] \times [8 : 16]$:

1. search x in the primary tree $\rightarrow O(\log N)$ canonical nodes;
2. in each, a 1D range search on y in its associated structure.

Answer: Mobile, Atlanta – as in the book (p. 18).

2D range search (Samet pp. 16–18, procedure 2dSearch)



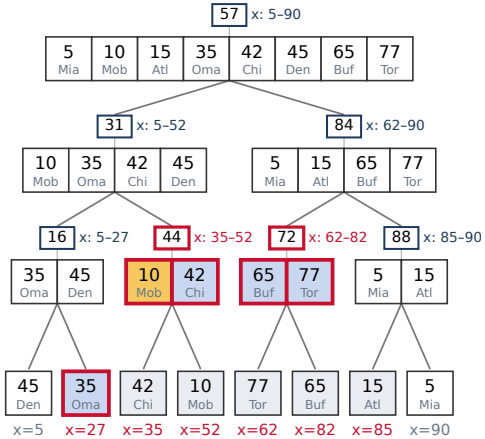
node {Mob, Chi}: y ∈ [8,16] → Mobile

Query $[25 : 85] \times [8 : 16]$:

1. search x in the primary tree $\rightarrow O(\log N)$ canonical nodes;
2. in each, a 1D range search on y in its associated structure.

Answer: Mobile, Atlanta – as in the book (p. 18).

2D range search (Samet pp. 16–18, procedure 2dSearch)



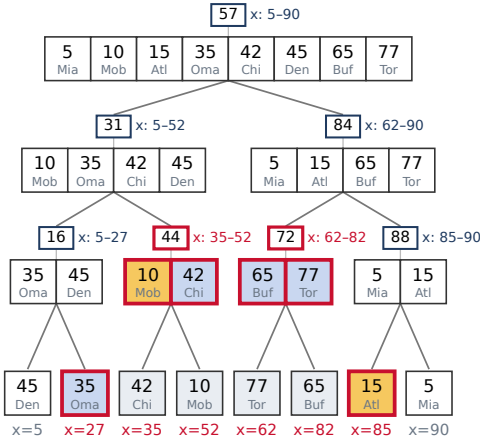
node {Buf, Tor}: y ∈ [8,16] → none

Query $[25 : 85] \times [8 : 16]$:

1. search x in the primary tree $\rightarrow O(\log N)$ canonical nodes;
2. in each, a 1D range search on y in its associated structure.

Answer: Mobile, Atlanta – as in the book (p. 18).

2D range search (Samet pp. 16–18, procedure 2dSearch)



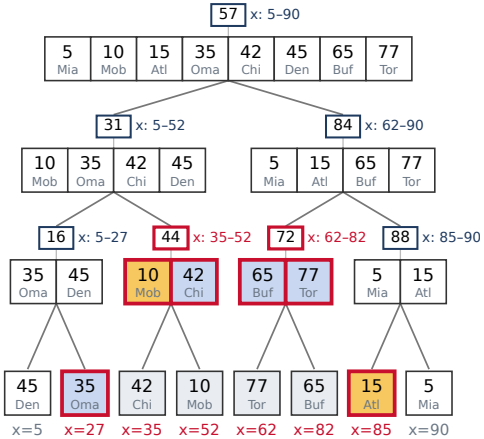
node {Atl}: y ∈ [8,16] → Atlanta

Query $[25 : 85] \times [8 : 16]$:

1. search x in the primary tree $\rightarrow O(\log N)$ canonical nodes;
2. in each, a 1D range search on y in its associated structure.

Answer: Mobile, Atlanta – as in the book (p. 18).

2D range search (Samet pp. 16–18, procedure 2dSearch)



answer: Mobile (52,10), Atlanta (85,15)

Query $[25 : 85] \times [8 : 16]$:

1. search x in the primary tree $\rightarrow O(\log N)$ canonical nodes;
2. in each, a 1D range search on y in its associated structure.

Answer: Mobile, Atlanta – as in the book (p. 18).

Analysis

	2D	why
space	$O(N \log N)$	each point is in one associated structure per level: $\lceil \log_2 N \rceil + 1$ copies
build	$O(N \log N)$	presort once; merge children's lists
query	$O(\log^2 N + F)$	$O(\log N)$ canonical nodes $\times$ one $O(\log N)$ search each, $+$ output

k **dimensions** (recursion on dimension): the associated structure of a node is a $(k-1)$-dimensional range tree.

$$\text{query } O(\log^k N + F), \qquad \text{space and build } O(N \log^{k-1} N).$$

Analysis

	2D	why
space	$O(N \log N)$	each point is in one associated structure per level: $\lceil \log_2 N \rceil + 1$ copies
build	$O(N \log N)$	presort once; merge children's lists
query	$O(\log^2 N + F)$	$O(\log N)$ canonical nodes $\times$ one $O(\log N)$ search each, $+$ output

k **dimensions** (recursion on dimension): the associated structure of a node is a $(k-1)$-dimensional range tree.

$$\text{query } O(\log^k N + F), \qquad \text{space and build } O(N \log^{k-1} N).$$

Worst-case guarantees, independent of the data distribution – paid for with $\log^{k-1} N$ extra space, and the structure is essentially *static*.